Report for the project in the course DD2438 at KTH

GROUP 27

Philipp Seidel Mo Kong

2000.08.30 2004.03.12

psei@kth.se mkong@kth.se

Abstract

Efficient autonomous navigation requires a delicate balance between global geometric optimality and local dynamic feasibility—a gap that traditional grid-based or purely stochastic planners often fail to bridge. This project develops a unified kino-dynamic navigation framework designed to generate physically executable trajectories in complex environments. We propose a hybrid hierarchical architecture: a coarse-grid A* planner provides global heuristic guidance, while a Movement Tree Planner, utilizing RRT* with a discrete library of movement primitives, ensures that the resulting paths respect the vehicle's inherent kinematics. To translate these plans into stable motion, we implement a predictive Coupled PD controller with look-ahead speed regulation for ground vehicles and a reactive ray-cast avoidance system for UAVs. Evaluated in Unity across diverse, cluttered terrains, our approach significantly reduces search redundancy compared to unguided sampling while maintaining smooth, collision-free execution. This work is important because it demonstrates that integrating heuristic biasing with dynamics-aware primitives can overcome the "jagged path" limitations of classical search and the computational inefficiency of standard sampling, providing a scalable pipeline for real-world autonomous systems.

1 Introduction

In the rapidly evolving landscape of autonomous robotics, the ability of vehicles to navigate complex environments safely and efficiently is a fundamental challenge. Whether for an Unmanned Aerial Vehicle (UAV) inspecting a forest or a ground vehicle navigating urban streets, the core problem remains the same: how to move from point A to point B without collision while adhering to the physical limitations of the machine. A path that looks optimal on a geometric grid is useless if the vehicle's kinematics prevents it from executing that path. Solving this problem effectively enables fully autonomous operations where human intervention is no longer required for complex maneuvering, significantly increasing the scalability and safety of robotic applications.

Mainstream grid-based algorithms, such as A* [4] , excel at finding optimal geometric paths but often fail to account for vehicle kinematics. By treating agents as holonomic points, they produce sharp, jagged trajectories that are practically impossible for non-holonomic vehicles to track. To address this, kino-dynamic RRT* [11] samples control inputs directly to ensure physical feasibility. However, due to their randomized nature, these methods frequently generate erratic, suboptimal paths and suffer from high computational costs when exploring vast state spaces without heuristic guidance.

In this project, we propose a unified navigation framework that synthesizes the strengths of these approaches while mitigating their respective weaknesses. We implement a Movement Tree Planner that adopts the kinematic validity of Dynamic RRT but replaces blind random sampling with a systematic, heuristic-guided expansion. Our planner utilizes a finite set of "Movement Primitives" – predetermined trajectory segments representing valid maneuvers. To overcome the inefficiency of standard sampling, we integrate an A*-based heuristic that biases the tree's growth toward the goal along an optimal path, significantly reducing the search space, and ensuring faster convergence to a solution.

To ensure that these feasible plans translate into smooth motion, we couple the planner with a robust execution layer: a Proportional-Derivative (PD) controller for the car that actively minimizes cross-track and heading errors, and a reactive, ray-cast-based avoidance system for the drone. Experimental results in a Unity simulation demonstrate that this hybrid approach generates smoother, drivable paths. It is also significantly faster than unguided sampling methods, effectively handling both the non-holonomic constraints of the car and the inertial dynamics of the drone.

1.1 Contribution

The primary contribution of this work is the development of a unified kino-dynamic navigation pipeline that guarantees path feasibility by incorporating vehicle dynamics directly into the search process. Unlike standard geometric planners that treat vehicles as holonomic points, our Movement Tree Planner expands the search space using discrete, physically valid movement primitives, ensuring that every generated path segment respects the specific acceleration and steering constraints of the vehicle. To mitigate the computational inefficiency typically associated with sampling-based methods, we integrated an A*-based heuristic that biases the tree growth toward the goal, significantly reducing the search space compared to unguided exploration.

Furthermore, this project contributes a robust execution layer designed to bridge the gap between simulation planning and real-time control. For the non-holonomic car, we implemented a Coupled PD Controller featuring a novel look-ahead speed regulation mechanism, which proactively decelerates the vehicle before sharp turns to minimize tracking errors and instability. For the drone, we developed a Reactive Avoidance System that fuses global path following with local collision prevention, utilizing multi-directional ray-casts to dynamically repel the agent from obstacles without requiring global re-planning. Collectively, these components constitute a complete system for reliable autonomous navigation in complex, obstacle-rich environments.

1.2 Outline

The report traces the development of our navigation framework from conceptual foundations to practical implementation and evaluation. In Section 2, we place our work in context by reviewing classical geometric planners, sampling-based and kino-dynamic approaches, as well as vehicle control strategies, highlighting the limitations that motivated our hybrid solution. Building on this, Section 3 presents the proposed approach in detail, describing the two-level planning architecture that integrates a coarse A*-based guidance path with movement-primitive-based RRT* for dynamic feasibility, alongside the control strategies for both car and drone and the modular software design that unites planning and execution. Section 4 then evaluates the system across diverse terrains and compares its performance with other groups. Challenges encountered are analyzed in Section 5, while Section 6 draws together the main insights and outlines directions for future improvements.

2 Related work

In the following we present the work that contributed and inspired our solution. We split this analysis into two. First we focus on the task of path planning, later on the subsequent task of path execution and vehicle control.

2.1 Path Planning

Autonomous navigation has historically been divided into geometric path finding and kino-dynamic planning, each with distinct trade-offs. The foundational A*-algorithm, introduced by Hart et al. [4], guarantees optimality through heuristic-guided search on a discrete grid. While powerful for holonomic agents, A* ignores differential constraints; when applied to car-like vehicles, it yields jagged trajectories that are practically undrivable without extensive post-processing.

To address high-dimensional configuration spaces where grid search fails, LaValle introduced the Rapidly-exploring Random Tree (RRT) [8]. This method efficiently explores free space via stochastic sampling but often produces suboptimal, erratic paths. Karaman and Frazzoli addressed this with RRT* [6] to guarantee asymptotic optimality. However, standard RRT* assumes straight-line connections, necessitating Webb and van den Berg's Kino-dynamic RRT* [11]. By solving Two-Point Boundary Value Problems (TPBVP), their approach ensures dynamic feasibility but introduces significant computational overhead, making real-time re-planning difficult.

To balance this feasibility with computational efficiency, Pivtoraiko et al. proposed State Lattice Planning [10], which discretizes the control space into a finite set of feasible maneuvers rather than random samples. Likhachev and Ferguson later demonstrated the power of combining these pre-computed lattices with heuristic search [9]. Drawing inspiration from this lattice-based philosophy, our Movement Tree Planner replaces the stochastic sampling of RRTs with a discrete library of "Movement Primitives". Movement primitives were inspired by neurobiology, where biological systems such as frogs and rats create complex muscle movements by combining basic motor commands [2]. By concatenating basic movements we can create complex trajectories. Guided by an A* heuristic, this approach ensures that generated plans are inherently smooth and executable, significantly accelerating convergence compared to purely randomized kino-dynamic methods.

2.2 Control Methods

Path tracking for non-holonomic vehicles traditionally relies on geometric methods. The Pure Pursuit algorithm [3] mimics human intuition by calculating curvature toward a look-ahead point, while the Stanley Controller [5] explicitly minimizes Cross-Track Error (CTE) and heading error via a non-linear control law. Although effective for lane-keeping, these methods often decouple steering from speed, leading to overshooting during high-speed cornering.

Alternatively, control-theoretic approaches employ feedback mechanisms established by Ziegler and Nichols [12] and reviewed by Ang et al. [1]. In vehicle navigation, PID strategies are widely used to minimize lateral deviations by dynamically adjusting steering based on accumulated error. However, standard PID implementations are inherently reactive, correcting errors only after they occur. Addressing this latency, our work extends the Coupled PD framework with a predictive look-ahead mechanism to proactively regulate speed before tracking stability is compromised.

For drone navigation in cluttered environments, the Artificial Potential Field (APF) method [7] offers a foundational reactive paradigm, modeling obstacles as repulsive forces within a vector field. However, global field calculation is computationally expensive and prone to local minima. Drawing on this concept, our system implements a lightweight, sensor-based approximation. We utilize multi-directional ray-casts to generate local repulsion vectors, effectively "pushing" the drone away from immediate hazards while maintaining its trajectory toward the goal.

3 Proposed method

In this section we present our approach in detail. We showcase our two-step path planning algorithm and explain the use of movement primitives to ensure dynamically feasible path generation. We also introduce our control strategy and explain both control algorithms for the drone and car simulations.

3.1 Path Planning

Our path planning utilizes a two-level approach. First we use A* [4] to plan a path on a simplified obstacle grid and then use kino-dynamic RRT* [11] based on movement primitives to construct a dynamically feasible path to the goal position. For this, the RRT* uses the A* path as a guide to get closer to an optimal solution and to reduce computational effort.

3.1.1 Preliminary A*

To provide an orientation for the RRT* we first compute a solution using A*. A* calculates an optimal path on a discrete grid using a heuristic to find the goal. In our approach this heuristic is simply the distance to the goal position. To reduce computational effort, this step is performed on a coarse grid with a cell resolution of 4×4. Because many terrains are cluttered with trees and other small obstacles that would block enough cells on a coarse grid to stop A* from finding a path, we first perform a opening operation on the obstacle grid. An opening operation consists of an obstacle deflation step followed by an obstacle inflation step. This eliminates small obstacles while keeping the larger obstacles in their original shape. The resulting path will therefore not necessarily avoid all obstacles, as can be seen in the example shown in Figure 1, but it will still be a reasonable guide for RRT* to find a better solution than by random exploration.

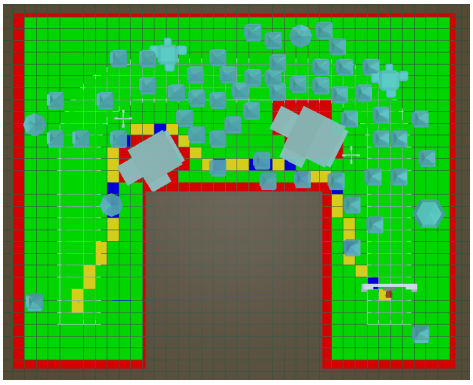

Figure 1: A* solution on simplified obstacle grid. Green: passable cell; Red: Blocked cell; Yellow: Path on passable cell; Blue: Path on cell that was blocked before the opening operation.

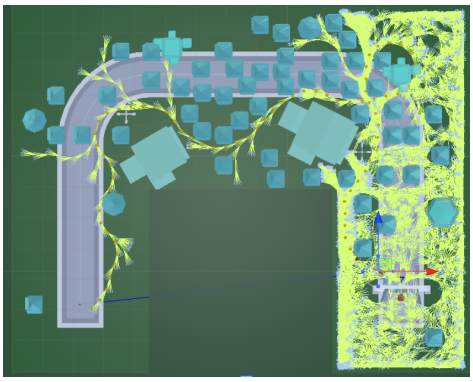

Figure 2: RRT* first struggles to follow the proposed path from A* as the terrain is very cluttered. It then reconnects to the guidance path to quickly grow towards the goal.

Figure 3: Two-Level path planning solution on terrain D

3.1.2 RRT* using Movement Primitives

RRT*, as first introduced by Karaman et al. [6], finds a path through an open space using random exploration as well as a heuristic. By sampling random points and growing towards them the tree produced by RRT will grow over the entire terrain. To guide RRT* towards the goal our implementation

chooses between random points and points on the A* path to grow towards. With a probability of 80% the next point that RRT* will grow towards will be a random point on the terrain. Otherwise the next point will be the closest point to the open set of RRT* on the A* path. The section of the A* path that is used for this is continuously updated every time RRT* grows close enough to the path. Because A* is working with a simplified grid, RRT* might not be able to closely follow the guidance path. By updating the section of the A* path that is used for guidance we allow RRT* to skip ahead and to reconnect to the A* path after deviating from it due to obstructions earlier. As illustrated in Figure 2, this allows RRT* to search for a solution in all the accessible space when needed, due to obstruction. After crossing the difficult part of the terrain RRT* then continues to closely follow the proposed A* path depicted in Figure 1.

To ensure dynamically feasible path generation, we utilize movement primitives for each simulation step. Each branch in the tree therefore represents a drivable trajectory. The path planner simulates each movement step using a dynamic model of the current vehicle to predict possible positions. To avoid obstacle collisions RRT* uses an obstacle grid with a cell size of 1×1. This obstacle grid undergoes obstacle inflation by the vehicle radius to ensure that any path on the grid will be collision free.

3.2 Vehicle Control

Ground vehicles and UAVs have fundamentally different kinematics. A ground vehicle is controlled via steering and longitudinal acceleration, tightly coupling velocity and orientation, while a UAV is omnidirectional, allowing decoupled movement relative to its heading. We adopt a simple bicycle model for planning, generating paths with position, heading, and speed for the controller. For cars, heading and speed are primary targets; for UAVs, position is the main reference.

3.2.1 Car Control

Longitudinal control uses a PD controller to track target speed from planned way-points. Lateral control integrates a PD regulator for heading and a secondary PD for Cross-Track Error (CTE) [5], ensuring alignment and positional accuracy. Way-point progress is monitored using a vector-based projection criterion, with the front axle as the reference point. A look-ahead strategy dynamically selects the minimum velocity over upcoming segments, enabling proactive deceleration for stability in turns.

3.2.2 Drone Control

The UAV controller directly commands accelerations along x and z axes. Position errors are decomposed into tangential and normal components relative to the desired path, producing corrective accelerations. Reactive obstacle avoidance is achieved using ray-cast-based artificial potential fields [7], generating repulsive vectors from nearby obstacles to maintain safe, collision-free trajectories.

3.3 Implementation

Our implementation achieves a high degree of modularity which allowed us to test a variance of implementations in quick succession. This architecture keeps our path planning and path execution fully separate as illustrated in Figure 4. They interact only by utilizing the shared 'Path' class. The Path-Planning package further separates the logic of path planning from our dynamics model. This high degree of separation allows for a trivial insertion of arbitrary dynamic models and streamlined the adaptation of our solution from car to drone dynamics.

Our movement primitive implementation is a set of input commands interpreted by the vehicle dynamics module. How the steering and acceleration held by a movement primitive are interpreted is fully up to the vehicle dynamics implementation. In our implementation the vehicle dynamics apply the movement primitive input commands in multiple successive discrete steps, approximating the continuous movement displayed by the modeled vehicle. The time difference between nodes is scaled by the current velocity and the number of discrete approximation steps to ensure a uniform distance between graph nodes. Therefore, at higher vehicle speeds, the time between nodes decreases to allow quicker reactions to obstacles.

For car control, to ensure smooth velocity transitions, the baseline target speed is linearly interpolated between the previous and current way-points based on the vehicle's progress ratio along the segment. The look-ahead mechanism is implemented with a specific Sharp Turn Logic: the system queries the path heading at a fixed index horizon. If the angular difference exceeds a threshold, the velocity limit is aggressively capped to ensure traction during high-curvature maneuvers. Additionally, lateral ray casts generate a reactive steering adjustment that is summed with the standard PD command, allowing the vehicle to locally nudge away from obstacles without requiring global re-planning.

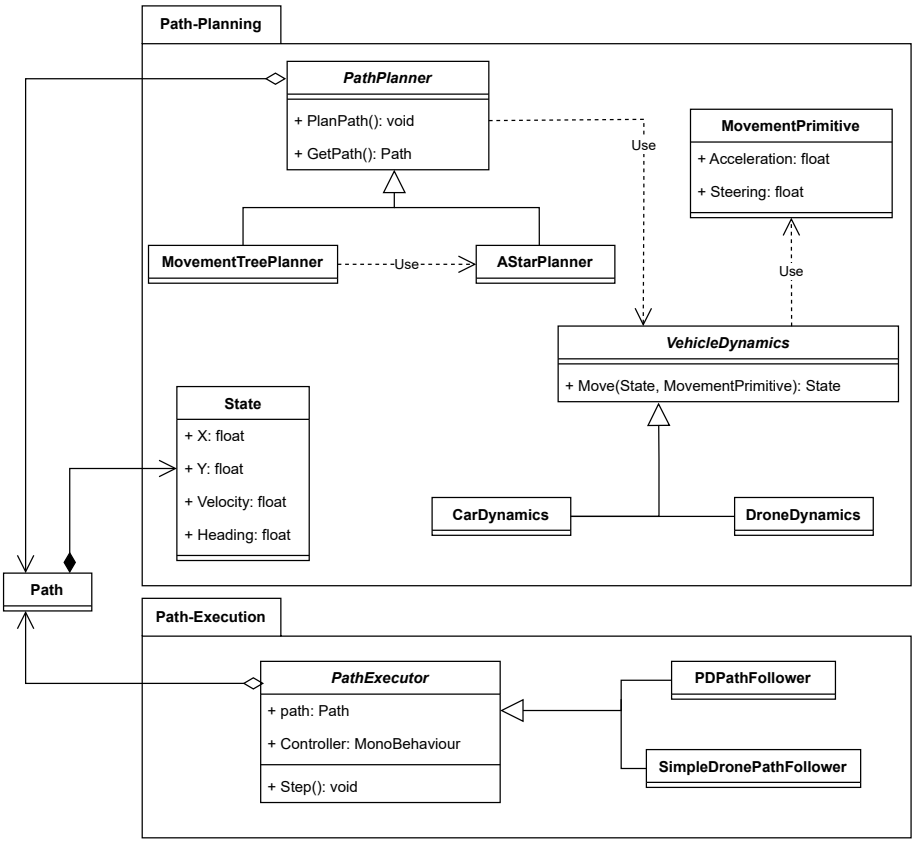

Figure 4: Package and class structure of both path planner and path executor

4 Experimental results

In this section we present the results achieved by our solution. We compare our results to those of the best performing group as well as to a group with a similar approach to ours. We analyze our performance and give reasoning for shortcomings.

4.1 Experimental setup

Simulations were conducted in Unity 6 LTS across ten terrains. The planner successfully generated trajectories in all but one terrain for the car (N2_hard) and one for the drone (L2). Planning 30,000 iterations required roughly 35 minutes on an Intel® Core™ i7-14650HX.

4.2 Analysis of Outcome

Simulation results indicate that the computational time required for path planning is heavily dependent on the scale and complexity of the environment. For instance, in the L2 terrain, a maze environment characterized by complex wall obstacles, path planning requires approximately 35 minutes. In contrast, on the smaller terrain D, the process completes in just 10 minutes.

This discrepancy is primarily attributed to the exploration mechanism within the algorithm. During the generation of each new node, the planner must iterate through all the terminal nodes of the search tree. Consequently, as the tree expands, the computational complexity increases linearly with the number of nodes, causing the growth rate to decelerate significantly.

Furthermore, the efficiency of the planner relies on the A*-based guidance path. When the A* planner identifies a geometrically valid guide path, the Movement Tree converges rapidly. However, if the A* guide is kinematically infeasible for the vehicle, the tree reverts to uniform sampling across the state space. This leads to an exponential increase in tree size and prevents the system from finding a solution within a reasonable time frame.

In all scenarios where a valid path was generated, our vehicle demonstrated smooth trajectory tracking with a small probability of collision. However, we observed a performance gap in terms of speed compared to the top-performing groups such as Group 17.

This difference can be attributed to two main factors. First, we did not implement path smoothing or post-optimization, which meant the generated trajectories were not always the kinematically optimal shortest paths. Second, our PD controller incorporates a look-ahead strategy that dynamically selects the minimum velocity among the current and subsequent five way-points as the control target. While this mechanism ensures proactive deceleration before high-curvature turns to maintain stability, it inevitably results in a lower average velocity.

Regarding the drone, our controller implements pure position tracking without an explicit target velocity profile. Consequently, the system prioritizes trajectory adherence over speed maximization. With the exception of the extremely narrow terrain L2, the drone demonstrated smooth, collision-free flight across all tested environments.

Group 2 adopted path planning and control algorithms similar to ours. Specifically, they implemented the RRT* algorithm, integrated with a Grid-based Variable Probability (GVP) sampling strategy to optimize path search efficiency. Furthermore, they also utilized a PD controller. The performance of both groups across the competition terrains is compared in Table 1.

It can be observed that their path planner exhibits lower environmental

	K2	L2	M2	N2_hard	N2	O2	O2_hard
Car	62.46	94.08	52.62	failed	—	—	116.32
Car	31.34	54.18	28.52	28.24	—	—	41.86
Car	30.82	57.6	25.96	18.42	—	—	failed
Drone	167.45	failed	142.32	—	59.92	341.8	—
Drone	41.66	75.02	39.86	—	16.14	45.46	—
Drone	70	failed	38.52	—	125	failed	—

Table 1: Results: Ours(black), Group 17 (orange), Group 2(blue)

adaptability compared to ours, particularly in scenarios with dense obstacles or narrow passages. For instance, they failed to generate valid trajectories in terrain L2 (characterized by narrow corridors between walls) and the urban terrains O2 and O2_hard (dense with trees and buildings). In contrast, our planner successfully generated a trajectory for terrain L2, although our drone controller was unable to execute it.

This performance gap stems from the fundamental differences in planning approaches. Their method generates raw paths as poly-lines, which necessitate aggressive smoothing algorithms. In extremely constrained spaces, this approach requires excessively dense sampling and smoothing to prevent the linearized path from intersecting walls. Conversely, our planner generates trajectories that inherently adhere to the vehicle's physical model (kino-dynamic constraints).

However, regarding speed, they outperformed us. An analysis of the controllers reveals that they implemented a detailed lookup table (dictionary) mapping curvature radius to specific maximum velocities, allowing their vehicle to maintain higher speeds during cornering. In comparison, our strategy relies on a conservative threshold: detecting a sharp turn above a specific angle and clamping the velocity to a low value. Furthermore, their RRT* algorithm continuously rewires tree nodes to minimize path cost, resulting in trajectories that are predominantly straight lines, which allows for faster execution.

5 Analysis of Failure

Our solution failed to produce trajectories in two competition terrains. On terrain N2_hard our path planner for the car did not find a viable path to the goal. On terrain L2 our drone controller could not fully execute the calculated path. In the following, we analyze why these problems occurred and why we could not prevent them at an earlier stage in the development process.

5.1 Technical Aspect of Failure

5.1.1 Car on terrain N2_hard

Our path planner could not find a viable path through the terrain within our set iteration limit. This is likely due to the very small distances between the trees in this terrain. Our RRT* path planner uses an obstacle grid to check for possible collisions. In this obstacle grid the obstacles are inflated by an approximation of the car radius. With very small obstacle distances, however, this approach leads to impenetrable walls of blocked cells in the obstacle grid, which prevents a path from reaching the goal position. We attempted to mitigate this problem by reducing the used inflation radius. This led to a successful path generation. The drawback was that this path was much harder for our car controller to follow and frequently led to the car getting stuck due to unforeseen collisions. To address this we adjusted the collision prediction method of the controller to use a bounding box check. This led to a successful trajectory that reached the goal in 42 seconds. This result however did not qualify for the competition as it was only achieved shortly after the deadline.

5.1.2 Drone on terrain L2

On terrain L2 our path planner provided a satisfactory path. Our drone controller could not fully execute this path due to the extremely thin pathways present in the terrain. The drone controller tries to keep a safe distance to obstacles by applying a repelling force onto its control input. This force becomes stronger for smaller distances to the obstacles. Due to this the drone was unable to enter the thinnest pathway in terrain L2 because the walls of the pathway generated a strong enough repelling force to block the drone. We tried to mitigate this by dynamically recognizing this situation, based on prolonged low velocities. As a result the repelling force would be lowered until the path following component of the controller could overpower the obstacle forces. This did not lead to the expected result as the drone would

then impact the walls shortly after overcoming the obstacle forces, which led to it getting stuck.

5.2 Management Aspect of Failure

		Week number		Final Meeting	Report Handin
Group 27		**4**	**5**	**6**	**7**
	Planned Effort (accumulated)	25	50	75	100
Philipp Seidel	**Actual Effort (accumulated):**	10	24	64	91
Mo Kong	**Actual Effort (accumulated):**	11	22	55	85
	Actual Progress:	20	40	75	95
	Risk of Failure:	4	4	4	
	Comments on Risk:	(only needed if >5%)			
	Comments on Progress:	Needed only little time for terrain Z, and had little time last week, will catch up			
	Project management:	Agile development			

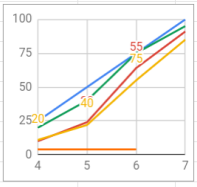

Figure 5: Our progress report time documentation and risk analysis

Our development approach for this project was to separate the development of the path planning and the path execution as much as possible. We did this in order to allow for a high degree of parallelization and less co-dependency. We met regularly to synchronize our work and to ensure a coherent result. This separation however also led to a reduced number of integration tests, most of which we performed only late into the development process. As can be seen in Figure 5 we continually, dramatically underestimated the risk of failure due to this. We assumed that the integration of our controllers and path planning approach would be more smooth and we did not anticipate to face as many problems at a late stage in development.

5.3 Lessons Learnt

We should have made sure to regularly check the full functionality of our solution. This would have allowed us to spot these late-stage problems and to address them well in advance. Both problems that caused our solution to finally fail are solvable within our framework. If we had spotted these problems earlier we could have prevented failure on two maps and likely improved performance on the rest. We also learned that a more streamlined communication is crucial to spot these kind of problems. We did not sufficiently coordinate our approaches and worked with assumptions about each others work that finally turned out to be flawed. Clearer, earlier definitions of interfaces and a clear time-line would have helped us to structure our approach better.

6 Summary and Conclusions

In this project we designed and implemented a modular path-planning and path-execution framework that could solve most of the competition terrains. We demonstrated an A*-based path planning guide for RRT* and successfully used movement primitives to generate dynamically feasible paths through complicated terrains. While our approach fell short of generating trajectories for all competition terrains, we have identified the technical problems present in our implementation. We are convinced that this framework can be adapted to solve all terrains with little additional effort and that it represents an innovative and practical foundation for future expansion. Future work might include the introduction of a path-smoother and the addition of a velocity optimizing drone controller.

References

[1] Kiam Heong Ang, Gregory Chong, and Yun Li. Pid control system analysis, design, and technology. *IEEE transactions on control systems technology*, 13(4):559–576, 2005.

[2] Emilio Bizzi, Ferdinando A. Mussa-Ivaldi, and Simon Giszter. Computations Underlying the Execution of Movement: A Biological Perspective. *Science*, 253(5017):287–291, July 1991. Publisher: American Association for the Advancement of Science.

[3] R Craig Coulter. Implementation of the pure pursuit path tracking algorithm. Technical report, 1992.

[4] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[5] Gabriel M Hoffmann, Claire J Tomlin, Michael Montemerlo, and Sebastian Thrun. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing. In *2007 American control conference*, pages 2296–2301. IEEE, 2007.

[6] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The international journal of robotics research*, 30(7):846–894, 2011.

[7] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The international journal of robotics research*, 5(1):90–98, 1986.

[8] Steven LaValle. Rapidly-exploring random trees: A new tool for path planning. *Research Report 9811*, 1998.

[9] Maxim Likhachev and Dave Ferguson. Planning long dynamically feasible maneuvers for autonomous vehicles. *The International Journal of Robotics Research*, 28(8):933–945, 2009.

[10] Mihail Pivtoraiko and Alonzo Kelly. Generating near minimal spanning control sets for constrained motion planning in discrete state spaces. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3231–3237. IEEE, 2005.

[11] Dustin J Webb and Jur Van Den Berg. Kinodynamic rrt*: Asymptotically optimal motion planning for robots with linear dynamics. In *2013 IEEE international conference on robotics and automation*, pages 5054–5061. IEEE, 2013.

[12] John G Ziegler and Nathaniel B Nichols. Optimum settings for automatic controllers. *Transactions of the American society of mechanical engineers*, 64(8):759–765, 1942.